

Gen AI Developer Task

Objective

Design and implement a **custom semantic document search engine** that finds the most relevant text documents for a user query — **without using any pretrained embeddings, NLP APIs, or external AI tools.**

Problem Description

You are to build a small backend application that accepts a search query and returns the top 3 most relevant text documents from a given corpus.

- You will be given a folder named `/documents/` containing **50 text files** (each with 300–500 words).
- The API should accept a query such as:

```
GET /search?q=artificial intelligence in finance
```

- The service should compute similarity between the query and each document, rank them, and return:
 - The **top 3 document filenames**
 - Their **similarity scores**
 - A short **text snippet** (first 2 lines or 200 characters)

Requirements

1. Custom Text Vectorization

- You must implement **TF-IDF or bag-of-words vectorization manually** — not using `sklearn`, `gensim`, or other NLP vectorizers.

- You can use only basic Python/Numpy operations (or equivalent in Node.js).

2. Similarity Computation

- Implement **Cosine Similarity** manually to compare vectors.
- Compute similarity between the user query vector and each document vector.

3. Backend Implementation

- Use either:
 - **Python (FastAPI / Flask)**
 - **Node.js (Express)**
- API endpoint: `/search?q=<query>`
Response format (example):

```
{  
  
  "results": [  
  
    {  
  
      "document": "finance_ai.txt",  
  
      "score": 0.87,  
  
      "snippet": "AI systems are transforming investment research by  
automating..."  
    },  
  
  ]  
}
```

- Bonus: Add `/index` endpoint to reprocess the document folder if new files are added.

4. Deliverables

- Source code (structured & modular)
- `README.md` with:
 - Setup instructions
 - Sample API calls
 - Example responses
- (Optional) Minimal UI (HTML or React) that lets users type queries and view ranked results.

Constraints

- No external AI/NLP libraries (e.g., Hugging Face, Sentence Transformers, Spacy, Gensim, Scikit-learn TF-IDF).
- Use only standard libraries like `math`, `numpy`, or `re` (for tokenization).
- Implement all logic manually for text processing and similarity.